Software Requirements Specification (SRS)

Project Title: Retirement Planning Tool
Course: SENG472 – Term Project
Team No: 1
Team Members:

Onat Ilgaz Keser - Technical Architect

Muhammet Furkan Çam - Quality Assurance Specialist

Mahmut Emre Demir, - Requirements Engineer

Adem Aşkan, - Project Manager

Ekrem Elbasan - System Analyst

Date: *27.04.2025*

Software Requirements Specification (SRS) 1
1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Definitions, Acronyms, and Abbreviations 3
2. Functional Requirements 4
2.1 Main Functions and Services 4
2.1.1 Conversational Health Profile Management 4
2.1.2 Retirement Sustainability Assessment 5
2.1.3 Financial Status Analysis 5
2.1.4 Healthcare Cost Conversation 5
2.1.5 Investment Recommendation System 5
2.2 Use Cases 5
2.3 Data Requirements (with Real-World Datasets) 7
2.3.1 User-Provided Data 8
2.3.2 Externally Sourced Data 8
3. Non-Functional Requirements 9
3.1 Performance Requirements 9
3.2 Security Requirements 9
3.3 Usability Requirements 10
4. System Architecture 10
4.1 High-Level Design 10
4.2 Required Technologies, Libraries, and Frameworks (Expanded) 11
4.2.1 Technologies 11
4.2.2 Libraries and Frameworks 12
5. Implementation Plan 12
5.1 Milestones and Timeline 12
5.2 Task Assignment 13
6. Evaluation and Testing 13
6.1 Testing Plan 13
6.2 Success Metrics 15
7. Conclusion 16
8. Appendix 17

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to detail the requirements for the Retirement Planning Tool. This document outlines the functional and non-functional requirements necessary to develop a software application that assists users in planning for retirement. The tool aims to provide personalized insights and recommendations by integrating various factors, including financial status, health profile, longevity risk, and market conditions. It leverages advanced AI/LLM technologies to offer a user-friendly, conversational interface for users across different financial literacy levels.

1.2 Scope

The scope of the Retirement Planning Tool is to provide a comprehensive, data-driven, and personalized retirement planning experience. The system will include the following core capabilities:

- Collecting and managing user health profile data through conversational interaction.
- Analyzing user financial status (savings, income, expenses) via a guided dialogue.
- Generating and explaining personalized retirement age recommendations and projections based on provided data.
- Projecting potential healthcare costs in retirement.
- Performing longevity risk analysis to assess the probability of outliving savings.
- Suggesting tailored investment options and pension schemes.
- Providing a user-friendly conversational chatbot interface.
- Assessing retirement plan sustainability against desired lifestyle.
- Integrating financial and health data from trusted external sources.

The system is intended for a target audience including young people starting retirement planning, middle-aged adults optimizing strategies, pre-retirees validating readiness, and financial advisors seeking advanced client tools. The initial focus will be on a core set of features, with potential for future expansion based on user feedback.

1.3 Definitions, Acronyms, and Abbreviations

- **AI:** Artificial Intelligence
- **API:** Application Programming Interface

- **ASPP:** Annual Survey of Public Pensions
- **BM25:** Best Match 25 (a ranking function used in search)
- **ChromaDB:** A vector database used for storing embeddings
- **CoT:** Chain-of-Thought Prompting
- **Gradio:** A Python library for building user interfaces for machine learning models
- **HMD:** Human Mortality Database
- **Hugging Face Spaces:** A platform for hosting machine learning demos and applications
- **LangChain:** A framework for developing applications powered by language models
- **LLM:** Large Language Model
- **NCHS:** National Center for Health Statistics
- **OAuth 2.0:** An authorization framework
- **PPD:** Public Plans Database
- **RAG:** Retrieval-Augmented Generation
- **RBAC:** Role-Based Access Control
- **ReAct:** Reasoning + Acting (Prompting technique)
- **SRS:** Software Requirements Specification
- **Vector Database:** A database optimized for storing and querying vector embeddings

---

2. Functional Requirements

2.1 Main Functions and Services

2.1.1 Conversational Health Profile Management

The system shall allow users to provide and manage their health information through a natural language conversational interface.

- **2.1.1.1 Health Data Collection:** The system shall collect user health data (e.g., current health status, lifestyle factors, relevant medical history) through natural language conversations.
- **2.1.1.2 Follow-up Questioning:** The system shall ask relevant follow-up questions to gather complete and necessary health information.
- **2.1.1.3 Health Data Update:** The system shall allow users to update previously provided health information through subsequent conversations.

2.1.2 Retirement Sustainability Assessment

Provides a personalized analysis to determine if the user's retirement savings and investment strategy will maintain their desired lifestyle throughout their projected lifespan.

Usefulness: Helps users identify potential financial shortfalls early and take corrective action to ensure long-term financial security during retirement.

2.1.3 Financial Status Analysis

Application gathers necessary information from user the assess the sustainability of the possible retirement plans. This financial information includes but is not limited to:

- savings
- income
- expenses
- properties (car, house, etc.)

The application may save the financial information in the database to use the information later.

2.1.4 Healthcare Cost Conversation

- Discuss potential healthcare expenses based on health profile and demographic information.
- Ask targeted follow-up questions about family medical history to improve predictions
- Explain healthcare cost factors in simple, understandable language

2.1.5 Investment Recommendation System

- Suggest appropriate pension schemes and investment options through dialogue
- Explain investment recommendations with rationale
- Adjust suggestions based on user feedback during conversation

2.2 Use Cases

1. Register

- Actors: User
- Scenario: User creates a new account in the system.

2. Login

- Actors: User
- Scenario: User authenticates to access their account.

3. Request retirement age estimation

- Actors: User
- Scenario: User receives personalized retirement age recommendation based on their data.

4. Input health data

- Actors: User
- Scenario: User enters personal and family health information.

5. See possible retirement plans

- Actors: User
- Scenario: User views different retirement scenarios based on their financial and health data.

6. Add user specific documents

- Actors: User
- Scenario: User adds new personal documents about retirement planning information.

7. Remove user specific documents

- Actors: User
- Scenario: User removes a personal document from database.

8. Review risk calculations

- Actors: User
- Scenario: User examines longevity risk assessment and investment risk profiles.

9. Remove document

- Actors: Admin
- Scenario: Admin deletes documents from the system database.

10. Add document

- Actors: Admin
- Scenario: Admin uploads new documents to the system.

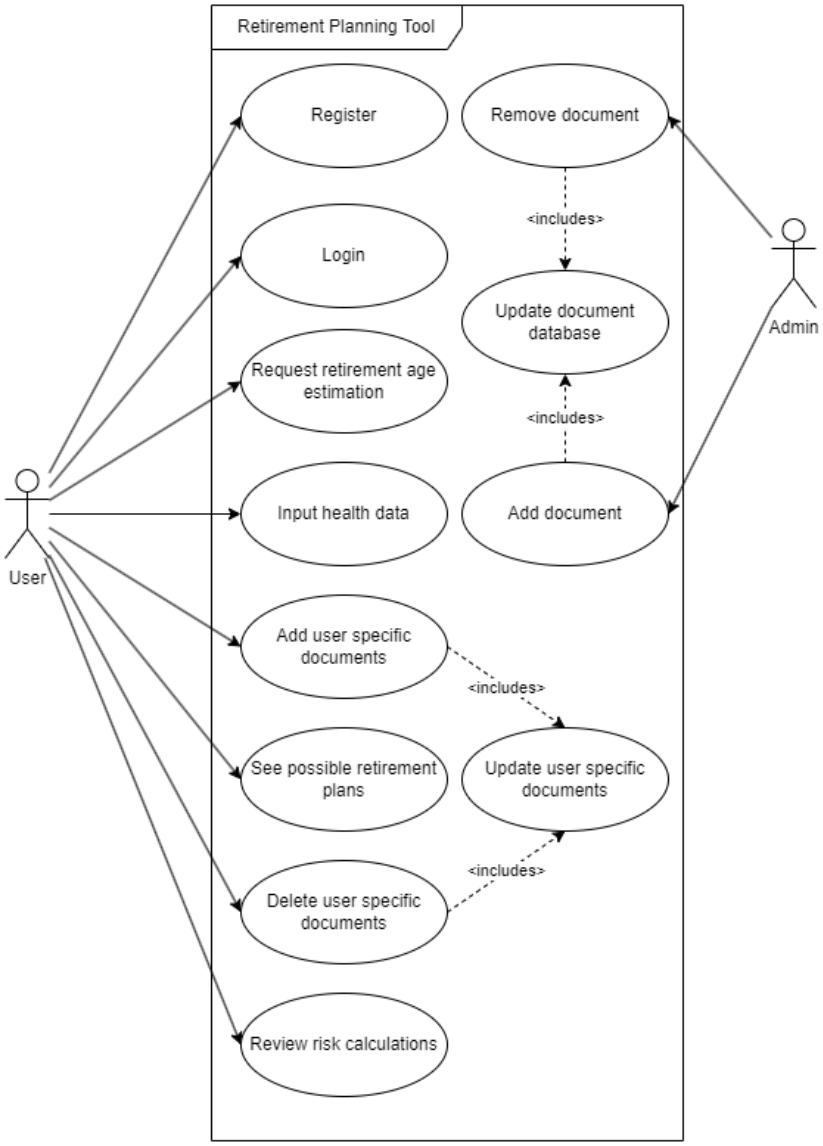


Figure 1

2.3 Data Requirements (with Real-World Datasets)

The Retirement Planning Tool uses a set of data requirements which are categorized into two groups: **User-Provided Data**, **Externally Sourced Data.**

2.3.1 User-Provided Data

Health Profile Data:

- **Description**: Information regarding the user's current health status, lifestyle choices, and medical history.
- **Example**: A user inputs that they have Type 2 diabetes, engage in moderate exercise thrice a week, and have a family history of heart disease.

Financial Data:

- **Description**: Details about the user's financial situation, including savings, income, expenses, and assets.
- **Example**: A user reports $75,000 in retirement savings, a monthly income of $4,000, monthly expenses totaling $2,500, and ownership of a home valued at $250,000.

Personal Data:

- **Description**: Demographic and employment information such as age, marital status, number of dependents, education level, and occupation.
- **Example**: A 50-year-old married individual with two children, holding a bachelor's degree, and employed as a software engineer for 25 years.

2.3.2 Externally Sourced Data

Financial Market Data:

- **Purpose**: To inform investment recommendations and retirement projections based on current market conditions.
- **Sources**:
  - Bloomberg API
  - Yahoo Finance API

Healthcare Cost Data:

- **Purpose**: To estimate potential healthcare expenses during retirement.
- **Sources**:
  - Fidelity Investments' Retiree Health Care Cost Estimate
  - Centers for Medicare & Medicaid Services (CMS) National Health Expenditure Data

Mortality and Longevity Data:

- **Purpose**: To assess longevity risk and help in planning the duration of retirement funds.
- **Sources**:
  - Human Mortality Database (HMD)
  - Social Security Administration (SSA) Life Tables

Public Pension and Social Security Data:

- **Purpose**: To incorporate expected benefits from public pension plans into retirement planning.
- **Sources**:
  - Social Security Administration (SSA)
  - Public Plans Database (PPD)

3. Non-Functional Requirements

3.1 Performance Requirements

- The system shall respond to user queries within 3 seconds during normal operation.
- The system shall maintain at least 98% uptime during operational periods.
- The chatbot shall maintain contextual awareness across multi-turn conversations without degradation in performance.

3.2 Security Requirements

- Sensitive user data, including health and financial information, shall be encrypted during transmission using TLS (Transport Layer Security) protocols.
- User authentication shall be managed securely through OAuth 2.0 authorization framework.
- The system shall implement Role-Based Access Control (RBAC) to restrict access to sensitive data based on user roles.
- Privacy controls and transparent data usage policies shall be provided to users before data collection.

3.3 Usability Requirements

- The conversational interface shall be intuitive and accessible for users with varying levels of financial literacy.
- The user interface shall provide simplified explanations of complex retirement and investment concepts.
- The system shall support both guided conversation paths (predefined flows) and user-initiated paths (freeform queries).
- The platform shall be accessible through mobile-friendly web interfaces to support usability across devices.

---

4. System Architecture

4.1 High-Level Design

1. User Interface Layer (Frontend)

- Gradio Interface: Provides the interactive UI for users to input their information and visualize results
- Authentication Module: Handles secure user login and session management
- Conversation Management: Facilitates the natural language conversation flow with users

2. Application Layer (Backend)

- Python Backend Service: Orchestrates the flow between frontend, AI models, and databases
- API Integration Manager: Connects to external financial and health data sources
- Data Processing Engine: Transforms and normalizes input data for analysis

4. Data Layer

- ChromaDB Vector Database: Stores embeddings of financial documents and health information
- Finance API: Retrieves necessary real time financial data.

5. LLM Agents

- Retirement Age Calculator: Determines optimal retirement age based on multiple factors
- Health Cost Predictor: Projects healthcare expenses based on health profile
- Longevity Risk Analyzer: Evaluates the risk of outliving savings

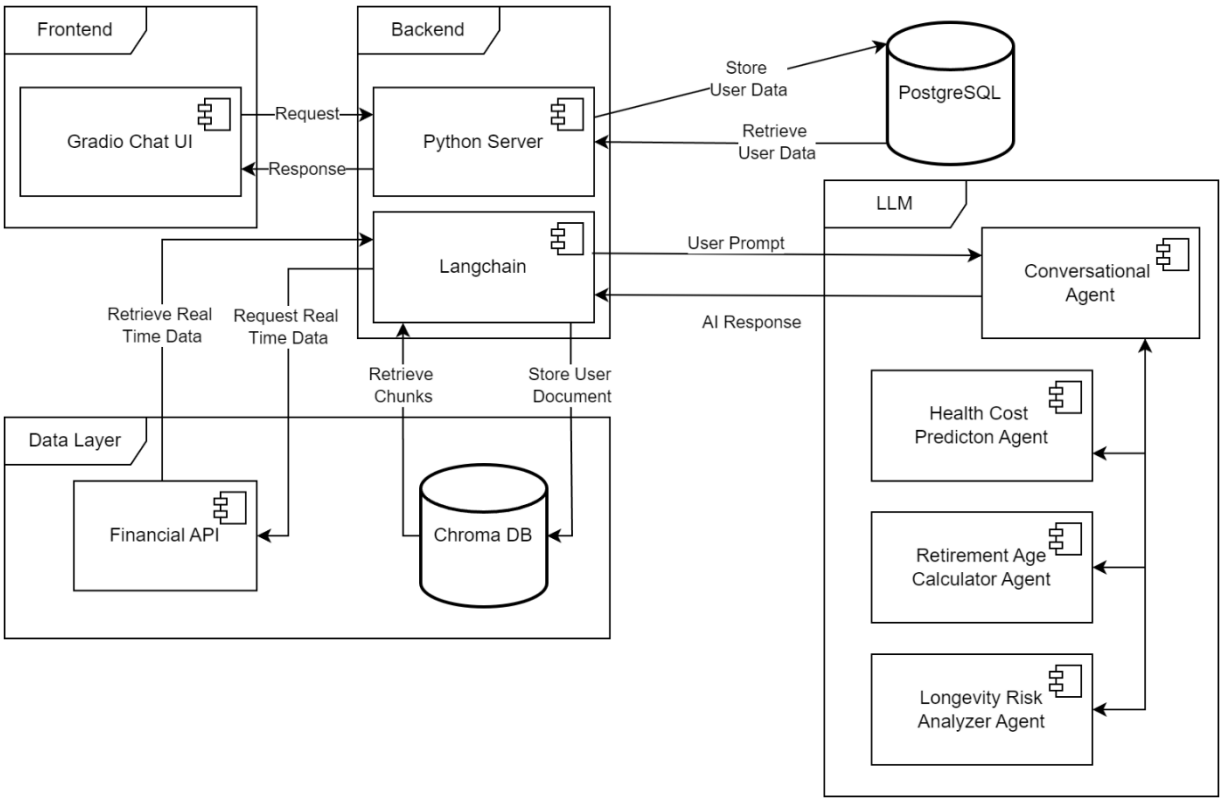


Figure 2

4.2 Required Technologies, Libraries, and Frameworks (Expanded)

To build and deploy the Retirement Planning Tool efficiently, the following **technologies, libraries, and frameworks** will be utilized:

4.2.1 Technologies

- **Python**:
  - Core backend development language for business logic, API handling, and data processing.
- **Gradio**:
  - Building interactive, user-friendly web-based interfaces for conversation and data input.
- **Google Gemini API**:
  - Main Large Language Model (LLM) used for conversational reasoning, personalized advice, and response generation.
- **ChromaDB**:

    - Vector database for storing embeddings from financial documents and user data, enabling semantic search.
- **Hugging Face Spaces**:
    - Platform for hosting and deploying the application, supporting easy updates and model management.

4.2.2 Libraries and Frameworks

- **LangChain**:
    - Framework to structure LLM-powered applications, enabling prompt chaining, memory management, retrieval-augmented generation (RAG), and integration with vector databases.
- **pandas**:
    - Data analysis and manipulation, especially for processing financial and health data collected from users or external APIs.
- **scikit-learn**:
    - Machine learning models (e.g., for basic risk prediction, clustering user profiles, or regression analysis on financial trends).
- **FastAPI**:
    - Backend API service for quick, scalable REST API development to connect frontend, database, and external services.
- **OAuth 2.0 libraries (e.g., Authlib, OAuthlib)**:
    - Managing secure user authentication and authorization.
- **PyCryptodome**:
    - Encryption of sensitive data (financial, health, personal information) both at rest and during transit.
- **Plotly / Matplotlib**:
    - Generating interactive visualizations such as financial projections, retirement plan comparisons, and risk assessment charts.

5. Implementation Plan

5.1 Milestones and Timeline

1. Week 1: Finalize system design, use cases, and set up Gradio frontend and Python backend infrastructure.
2. Week 2: Integrate Google Gemini LLM with LangChain and implement core functionalities (health profile management, financial status analysis, retirement projection dialogue).

3. Week 3: Complete healthcare cost conversation, investment recommendation system, and ChromaDB integration; initiate functional and non-functional testing.
4. Week 4: Finalize full system testing, polish user interface, prepare documentation, and complete final project presentation.

5.2 Task Assignment

- **Onat Ilgaz Keser (Technical Architect)**:
  - Design and implement the system architecture
  - Set up the Gradio frontend interface
  - Configure ChromaDB vector database integration
  - Implement API connections to external financial data sources
- **Muhammet Furkan Çam (Quality Assurance Specialist)**:
  - Develop comprehensive testing plans for all system components
  - Create and execute test cases for conversational flows
  - Validate data processing accuracy across the system
  - Perform security testing and ensure GDPR/privacy compliance
- **Mahmut Emre Demir (Requirements Engineer)**:
  - Refine user requirements and update functional specifications
  - Design conversational flows for health and financial data collection
  - Create detailed documentation for APIs and system interfaces
  - Develop user feedback collection mechanisms
- **Adem Aşkan (Project Manager)**:
  - Coordinate team activities and manage project timeline
  - Facilitate communication between team members
  - Track milestone progress and deliverables
  - Manage project risks and dependencies
- **Ekrem Elbasan (System Analyst)**:
  - Implement Google Gemini and LangChain integration
  - Develop prompt engineering techniques (Few-Shot, CoT, ReAct)
  - Create the Retrieval-Augmented Generation (RAG) system
  - Design financial analysis algorithms for retirement calculations

---

6. Evaluation and Testing

6.1 Testing Plan

- **Functional Testing:** Verification of core features such as conversational data collection (health and financial), retirement projection calculations, healthcare cost estimation, longevity risk analysis, and investment recommendation generation. Testing

conversational flows and the accuracy of the generated projections and recommendations is paramount.

- **AI Model Performance Testing:** Evaluating the accuracy and relevance of responses generated by the LLM (Google Gemini + LangChain) in handling financial queries and providing tailored advice, especially considering prompt engineering and grounding techniques (RAG, CoT, ReAct).
- **Data Validation and Quality Testing:** Implementing and testing mechanisms for validating user-provided information through conversation and ensuring the quality and correct integration of data from external sources (financial APIs, mortality databases).
- **Non-Functional Testing:**
    - **Security Testing:** Ensuring transparent privacy controls, adherence to data usage policies, and implementation of modern security standards (OAuth 2.0, RBAC).
    - **Conversation Quality Testing:** Assessing the naturalness of dialogue flows, contextual awareness across multi-turn conversations, and smooth transitions between topics.
    - **Usability Testing:** Evaluating the intuitiveness and accessibility of the conversational interface for users with varying financial literacy levels, verifying simplified explanations, and supporting guided/user-initiated paths.
    - **Reliability Testing:** Verifying fallback mechanisms for failed API connections.
    - **Adaptability Testing:** Confirming the system's ability to support evolving conversations based on new information and remember user context across sessions.
- **Integration Testing:** Testing the seamless integration between the frontend (Gradio), backend (Python), LLM/AI framework, vector database (ChromaDB), and external financial APIs.
- **Scalability Testing:** Although the proposal mentions scalability as a feasibility factor, testing should ensure the system can handle anticipated user load and data growth, especially considering deployment on Hugging Face Spaces.

A detailed test plan document, including specific test cases, responsibilities, testing environments, and success criteria, will be developed based on these requirements. Testing will likely involve a combination of automated and manual testing approaches, including user acceptance testing with representatives from the target audience. The iterative development approach mentioned in the feasibility analysis suggests continuous testing throughout the development lifecycle.

6.2 Success Metrics

To ensure the effectiveness, technical robustness, and user satisfaction of the Retirement Planning Tool, the following measurable success metrics have been established. These criteria are directly supported by the underlying technologies and techniques such as Prompt Engineering, Grounding, Retrieval-Augmented Generation (RAG), and Real-Time Financial Data Integration.

User Satisfaction with Personalized Estimations

- **Target:** At least 90% of users should express satisfaction with the personalized retirement age estimations provided by the platform.
- **Technical Performance Target**:
- **Prompt Efficiency Rate:** Achieve a minimum 85% prompt effectiveness, measured by relevance scoring between prompt context and model response.

Investment Portfolio Alignment

- **Target: 95% of the recommended investment portfolios** must align with the users' declared risk tolerance (low, medium, or high).

- **Technical Performance Target:**
  Chain-of-Thought Coherence: Ensure 90% logical consistency in multi-step reasoning responses verified through automated CoT evaluation scripts.

Improved User Understanding

- **Target:** At least 85% of users should report that the tool improved their understanding or planning for retirement after their first interaction.

- **Technical Performance Target:**
  Dialogue Coherence Rate: Maintain a minimum 92% user conversation flow satisfaction, measured via post-session feedback forms.

System Responsiveness

- **Target: 95% of user queries** must be answered within under 3 seconds through the chatbot interface.

- **Technical Performance Target:**
  Average Latency: Keep API response time below 2.5 seconds in 90% of cases, based on system logs.

Platform Uptime and Stability

- **Target:** Maintain at least 98% uptime during operational periods.

- **Technical Performance Target:**
  Downtime Events: Limit critical downtime incidents to no more than 2 events per operational quarter.

Real-Time Financial Data Grounding

- **Target: 90% of investment suggestions** must be based on verified and up-to-date financial market data from trusted sources.

- **Technical Performance Target:**
  Grounding Accuracy: Ensure **95% RAG retrieval precision**, verified through semantic search evaluation and retrieval hit rate.

Data Integrity Assurance

- **Target:** Ensure **99% data consistency** between user input and database storage (PostgreSQL).

- **Technical Performance Target:**
  Validation Success Rate: Achieve at least 99.5% validation pass across all database operations and API transactions.

7. Conclusion

The Retirement Planning Tool project aims to deliver a comprehensive, personalized, and data-driven retirement planning experience for users across different financial literacy levels. By integrating financial status, health profile, longevity risk, and real-time market data, the system provides tailored retirement age estimations, healthcare cost projections, investment recommendations, and sustainability assessments through an AI-powered conversational interface.

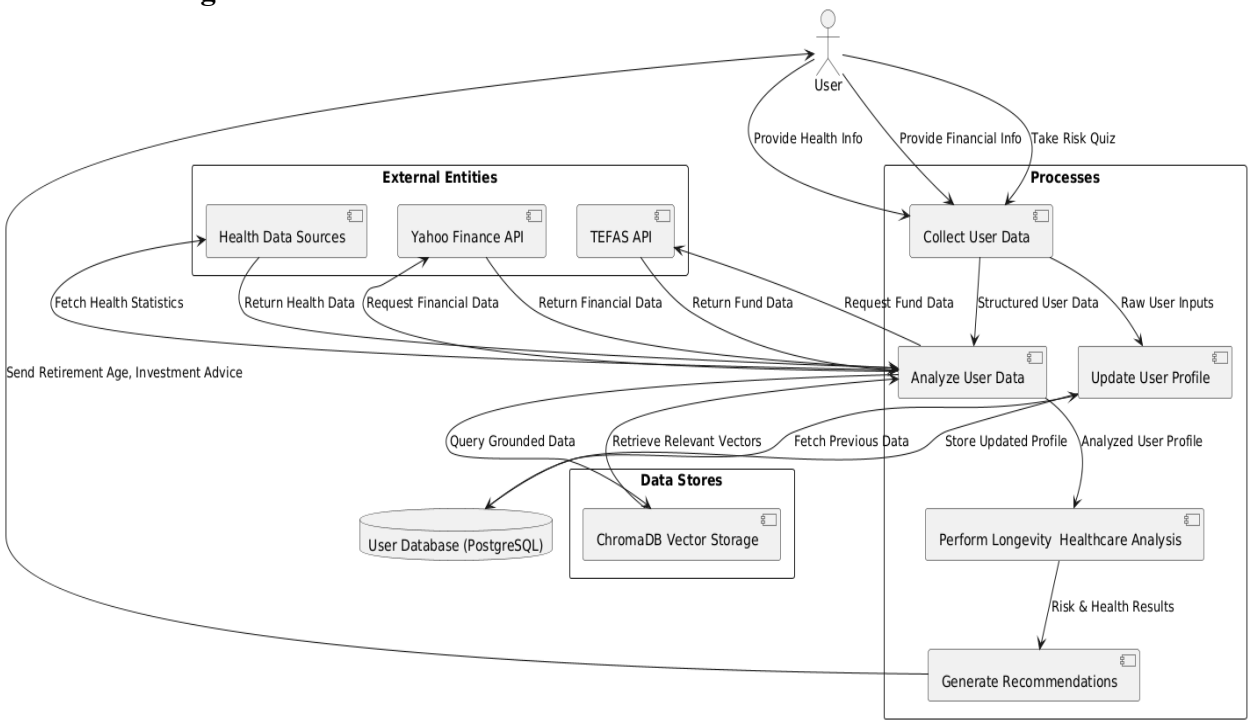
The expected outcomes of the project include empowering users to make informed and proactive retirement planning decisions, increasing early financial literacy and retirement awareness, and offering financial advisors a sophisticated, data-enriched tool to assist their clients. Furthermore, the dynamic adaptation of plans based on updated user data and real-time financial information positions the tool as a next-generation solution beyond traditional retirement calculators.

Potential challenges involve ensuring the continuous accuracy of real-time data integration, maintaining high levels of data privacy and security, and managing incomplete or inconsistent user input. Additionally, achieving high model response quality and maintaining grounding accuracy within financial recommendations are critical factors for success.

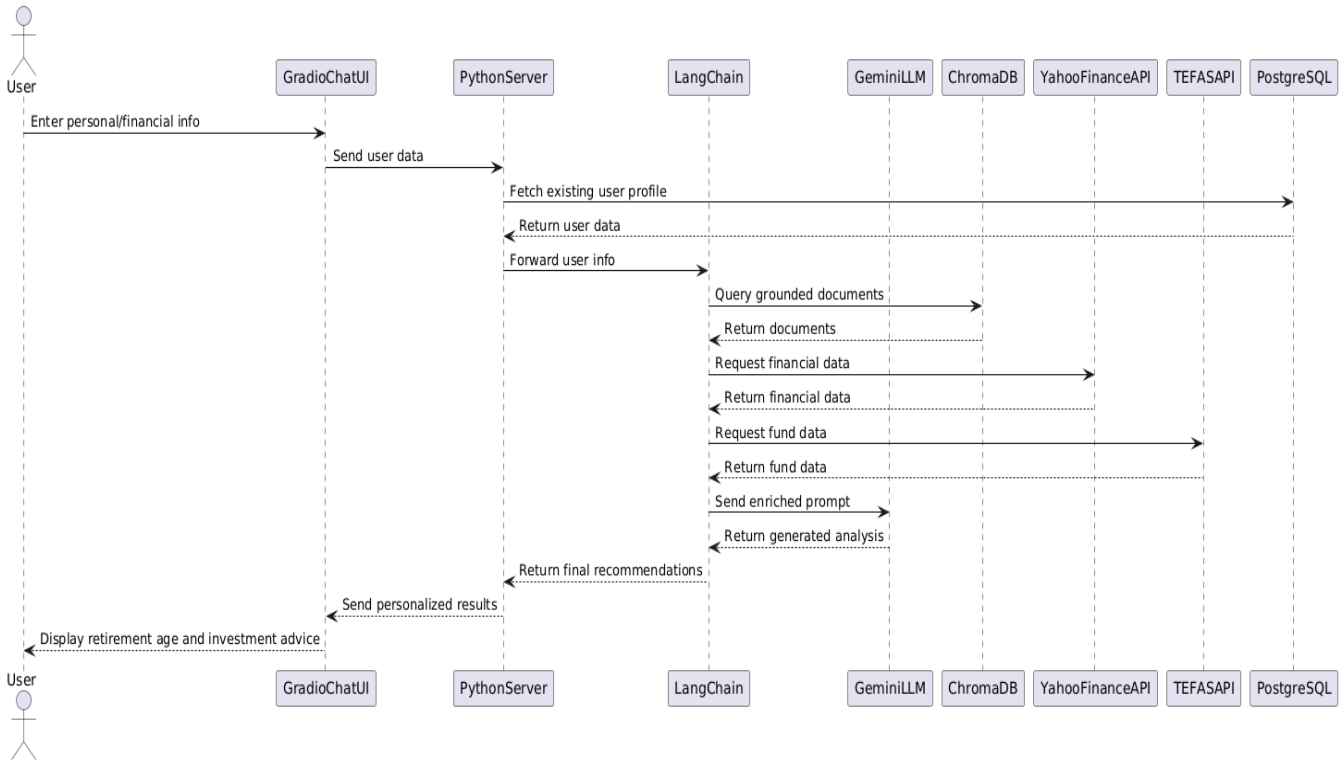
Despite these challenges, the combination of cutting-edge AI technologies (LLMs, RAG, Prompt Engineering) and robust system design provides a strong foundation for delivering a scalable and impactful solution in the retirement planning domain.

8. Appendix

Data Flow Diagram



Sequence Diagram


User
GradioChatUI
PythonServer
LangChain
GeminiLLM
ChromaDB
YahooFinanceAPI
TEFASAPI
PostgreSQL
Enter personal/financial info
Send user data
Fetch existing user profile
Return user data
Forward user info
Query grounded documents
Return documents
Request financial data
Return financial data
Request fund data
Return fund data
Send enriched prompt
Return generated analysis
Return final recommendations
Send personalized results
Display retirement age and investment advice
User
GradioChatUI
PythonServer
LangChain
GeminiLLM
ChromaDB
YahooFinanceAPI
TEFASAPI
PostgreSQL